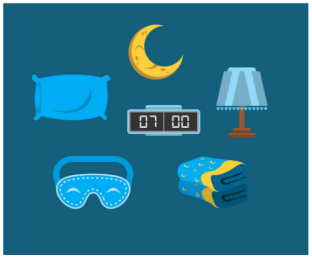
Welcome to Level 3 :-)

# A typical day in level 3



# Things to Do












# Things NOT to do





Welcome to CSE 314 :-)

# Outline

| Week | Lecture Topics | Corresponding CO(s) |
|---|---|---|
| Week 1 | Bash Lecture 1 & 2 | CO1 |
| Week 2 | Bash Lecture 1 & 2 + Bash Tutorial + Offline 1(Bash) Declaration | CO1 |
| Week 3 | RESERVE | |
| Week 4 | Offline 1(Bash) Submission + **Online 1(Bash)** + xv6 Lecture | CO1 and CO2 |
| Week 5 | **Online 1(Bash)** + xv6 Lecture + xv6 Tutorial + Offline 2 (xv6 System Call) Declaration | CO1 and CO2 |
| Week 6 | RESERVE | |
| Week 7 | Offline 2 (xv6 System Call) Submission + Offline 3 (IPC) Declaration + **Online 2 (xv6 System Call)** | CO1 and CO2 |
| | **Mid-break** | |
| Week 8 | **Online 2 (xv6 System Call)** | CO1 and CO2 |
| Week 9 | Offline 3 (IPC) Submission + **Online 3 (IPC)** + Offline 4 (xv6 Scheduling Declaration) | CO1, CO2, and CO3 |
| Week 10 | **Online 3 (IPC)** | CO1, CO2, and CO3 |
| Week 11 | RESERVE | |
| Week 12 | Offline 4 (xv6 Scheduling Submission) + **Online 4 (xv6 Scheduling)** | CO1 and CO2 |
| Week 13 | **Online 4 (xv6 Scheduling)** | CO1 and CO2 |
| Week 14 | Quiz | CO1, CO2, and CO3 |

through Continuous Improv

# CSE 314: Operating System Sessional

## Introduction and Shell Commands

Nafis Tahmid

Lecturer, CSE, BUET

# Why Linux?

# What is an Operating System?

- The software that sits between you and the hardware.

# What is an Operating System?

- The software that sits between you and the hardware.
- It manages the CPU, memory, disk, and devices for you.

# What is an Operating System?

- The software that sits between you and the hardware.
- It manages the CPU, memory, disk, and devices for you.
- Windows, macOS, Android, iOS . . . are all operating systems.

# What is an Operating System?

- The software that sits between you and the hardware.
- It manages the CPU, memory, disk, and devices for you.
- Windows, macOS, Android, iOS . . . are all operating systems.
- Linux is a free, open-source OS family that runs most of the world's servers, phones, and supercomputers.

# What is an Operating System?

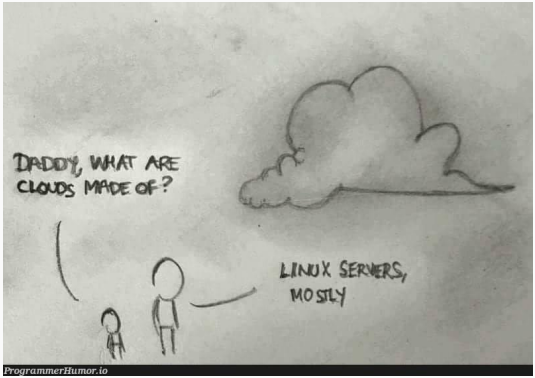
- The software that sits between you and the hardware.
- It manages the CPU, memory, disk, and devices for you.
- Windows, macOS, Android, iOS . . . are all operating systems.
- Linux is a free, open-source OS family that runs most of the world's servers, phones, and supercomputers.

Your lab PCs run Ubuntu, one of the most popular Linux distributions.

# Why should a CS student learn Linux?

# Why should a CS student learn Linux?

- It is what real servers, cloud machines, and most CS tools run on.



# Why should a CS student learn Linux?

- It is what real servers, cloud machines, and most CS tools run on.
- The command line lets you automate work that clicking can't.



# GUI vs. CLI

**GUI** (clicking)

- Easy tasks are easy

# GUI vs. CLI

**GUI** (clicking)

- Easy tasks are easy
- But you can only do what the buttons allow
- Hard to repeat or automate

# GUI vs. CLI

**GUI** (clicking)

- Easy tasks are easy
- But you can only do what the buttons allow
- Hard to repeat or automate

**CLI** (typing)

- The full power of the machine

# GUI vs. CLI

**GUI** (clicking)

- Easy tasks are easy
- But you can only do what the buttons allow
- Hard to repeat or automate

**CLI** (typing)

- The full power of the machine
- Difficult tasks become possible
- Easy to automate and repeat

# The Shell

# Shell and bash

- A shell is a program that reads your typed commands and runs them.

# Shell and bash

- A shell is a program that reads your typed commands and runs them.
- There are several shells; on Ubuntu the default is bash
  (**B**ourne **A**gain **SH**ell).

# Shell and bash

- A shell is a program that reads your typed commands and runs them.
- There are several shells; on Ubuntu the default is bash
  (**B**ourne **A**gain **SH**ell).
- You open it inside a terminal window.

# Shell and bash

- A shell is a program that reads your typed commands and runs them.
- There are several shells; on Ubuntu the default is bash
  (**B**ourne **A**gain **SH**ell).
- You open it inside a terminal window.
- Terminal lets you type commands; Bash interprets and runs them.

# Shell and bash

- A shell is a program that reads your typed commands and runs them.
- There are several shells; on Ubuntu the default is bash (**B**ourne **A**gain **SH**ell).
- You open it inside a terminal window.
- Terminal lets you type commands; Bash interprets and runs them.

Open one now: `Ctrl + Alt + T`

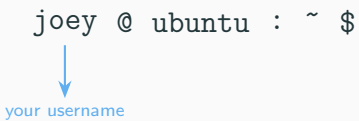
# Reading the prompt

Before you type anything, the shell shows a prompt:

```
joey @ ubuntu : ~ $
```

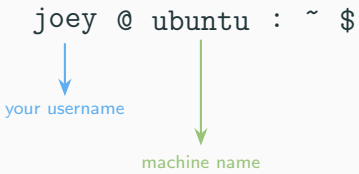
# Reading the prompt

Before you type anything, the shell shows a prompt:



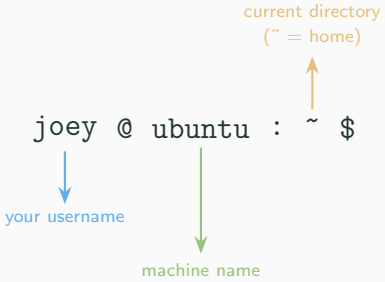
# Reading the prompt

Before you type anything, the shell shows a prompt:



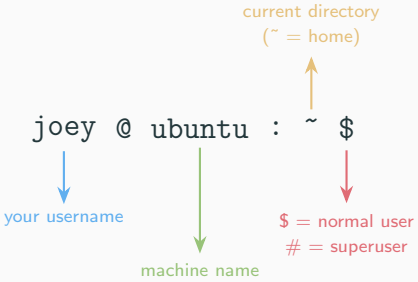
# Reading the prompt

Before you type anything, the shell shows a prompt:



# Reading the prompt

Before you type anything, the shell shows a prompt:



# Your first commands

Type a command, press `Enter`.

# Your first commands

Type a command, press `Enter`.

```
$ date
Fri Jun 19 10:30:14 +06 2026

$ whoami
joey

$ echo Hello world
Hello world
```

# Your first commands

Type a command, press `Enter`.

```
$ date
Fri Jun 19 10:30:14 +06 2026

$ whoami
joey

$ echo Hello world
Hello world
```

- `date` — shows the current date and time

# Your first commands

Type a command, press `Enter`.

```
$ date
Fri Jun 19 10:30:14 +06 2026

$ whoami
joey

$ echo Hello world
Hello world
```

- `date` — shows the current date and time
- `whoami` — prints your username

# Your first commands

Type a command, press `Enter`.

```
$ date
Fri Jun 19 10:30:14 +06 2026

$ whoami
joey

$ echo Hello world
Hello world
```

- `date` — shows the current date and time
- `whoami` — prints your username
- `echo` — prints back whatever you give it

# echo: arguments and quotes

- Words are separated by spaces; each word is an argument.

# echo: arguments and quotes

- Words are separated by spaces; each word is an argument.
- To keep spaces together, use quotes `" "` or a backslash `\`.

# echo: arguments and quotes

- Words are separated by spaces; each word is an argument.
- To keep spaces together, use quotes `" "` or a backslash `\`.

```
$ echo Hello       world
Hello world

$ echo "Hello       world"
Hello       world
```

# Handy shortcuts

- `Ctrl + Alt + T` : open terminal
- `Ctrl + L` : clear the screen

# Handy shortcuts

- `Ctrl + Alt + T` : open terminal
- `Ctrl + L` : clear the screen
- `Tab` : auto-complete names
- `Up / Down` : previous commands

# Handy shortcuts

- `Ctrl + Alt + T` : open terminal
- `Ctrl + L` : clear the screen
- `Tab` : auto-complete names
- `Up / Down` : previous commands
- `Ctrl + C` : cancel running command
- `Ctrl + A / E` : start / end of line

# Handy shortcuts

- `Ctrl + Alt + T` : open terminal
- `Ctrl + L` : clear the screen
- `Tab` : auto-complete names
- `Up / Down` : previous commands
- `Ctrl + C` : cancel running command
- `Ctrl + A / E` : start / end of line
- `Ctrl + Shift + C / V` : copy / paste

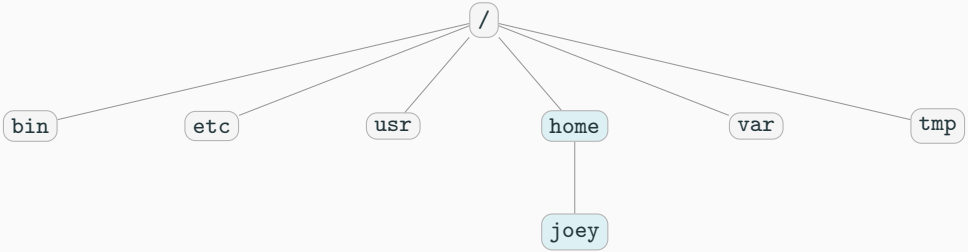
# Handy shortcuts

- `Ctrl + Alt + T` : open terminal
- `Ctrl + L` : clear the screen
- `Tab` : auto-complete names
- `Up / Down` : previous commands
- `Ctrl + C` : cancel running command
- `Ctrl + A / E` : start / end of line
- `Ctrl + Shift + C / V` : copy / paste

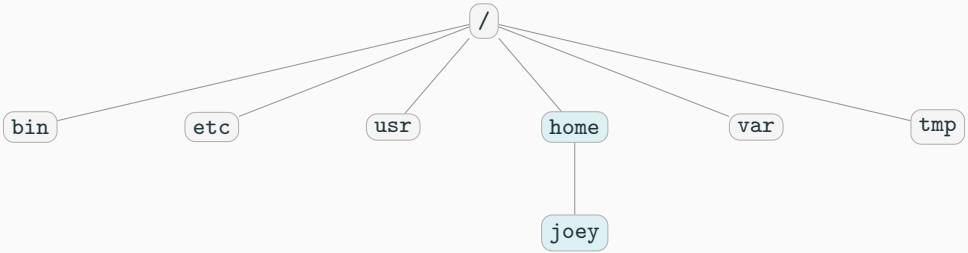
Tab is your best friend — it saves typing *and* prevents typos.

# Finding Your Way Around

# The filesystem is a tree

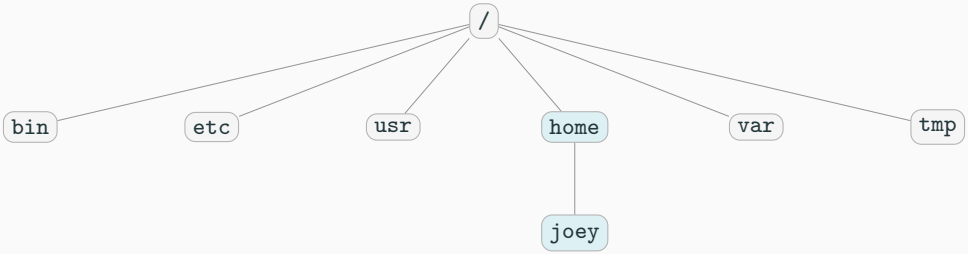


# The filesystem is a tree



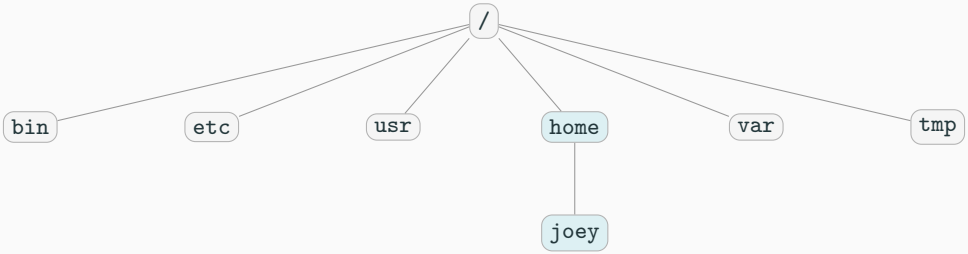

- `/` is the root — the top of everything.

# The filesystem is a tree




- `/` is the root — the top of everything.
- `/home/joey` is your personal folder, also written `~`.

# The filesystem is a tree




- `/` is the root — the top of everything.
- `/home/joey` is your personal folder, also written `~`.
- Everything branches down from `/`; folders separated by `/`.

# Where am I? pwd and cd

- `pwd` — **p**rint **w**orking **d**irectory (where you are now)

# Where am I? pwd and cd

- `pwd` — **p**rint **w**orking **d**irectory (where you are now)
- `cd PATH` — **c**hange **d**irectory (move somewhere)

# Where am I? pwd and cd

- `pwd` — **p**rint **w**orking **d**irectory (where you are now)
- `cd PATH` — **c**hange **d**irectory (move somewhere)

```
$ pwd
/home/joey

$ cd Documents
$ pwd
/home/joey/Documents
```

# Where am I? pwd and cd

- `pwd` — **p**rint **w**orking **d**irectory (where you are now)
- `cd PATH` — **c**hange **d**irectory (move somewhere)

```
$ pwd
/home/joey

$ cd Documents
$ pwd
/home/joey/Documents
```



# Where am I? pwd and cd

- `pwd` — **p**rint **w**orking **d**irectory (where you are now)
- `cd PATH` — **c**hange **d**irectory (move somewhere)

```
$ pwd
/home/joey

$ cd Documents
$ pwd
/home/joey/Documents
```

Plain `cd` (nothing after it) takes you back to your home.



# Absolute vs. relative paths

**Absolute path**

- Starts at the root `/`
- Full address, works from anywhere
- `/home/joey/code`

# Absolute vs. relative paths

**Absolute path**

- Starts at the root `/`
- Full address, works from anywhere
- `/home/joey/code`

**Relative path**

- Starts from where you are now
- `.` = current directory
- `..` = parent directory

# Absolute vs. relative paths

**Absolute path**

- Starts at the root `/`
- Full address, works from anywhere
- `/home/joey/code`

**Relative path**

- Starts from where you are now
- `.` = current directory
- `..` = parent directory

- `~` = your home directory
- `-` (a dash) = the previous directory you were in

# Absolute vs. relative paths

**Absolute path**

- Starts at the root `/`
- Full address, works from anywhere
- `/home/joey/code`

**Relative path**

- Starts from where you are now
- `.` = current directory
- `..` = parent directory

- `~` = your home directory
- `-` (a dash) = the previous directory you were in

Rule of thumb: use whichever is shorter when typing.

# Listing things: ls

`ls [options] [path]` — list what is inside a directory.

# Listing things: ls

`ls [options] [path]` — list what is inside a directory.

```
$ ls
Documents  Downloads  code  notes.txt
```

# Listing things: ls

`ls [options] [path]` — list what is inside a directory.

```
$ ls
Documents  Downloads  code  notes.txt
```

- `ls -a` — also show hidden files (names starting with `.`)

# Listing things: ls

`ls [options] [path]` — list what is inside a directory.

```
$ ls
Documents  Downloads  code  notes.txt
```

- `ls -a` — also show hidden files (names starting with `.`)
- `ls -l` — long, detailed listing

# Listing things: ls

`ls [options] [path]` — list what is inside a directory.

```
$ ls
Documents  Downloads  code  notes.txt
```

- `ls -a` — also show hidden files (names starting with `.`)
- `ls -l` — long, detailed listing
- `ls -la` — combine them

# Listing things: ls

`ls [options] [path]` — list what is inside a directory.

```
$ ls
Documents  Downloads  code  notes.txt
```

- `ls -a` — also show hidden files (names starting with `.`)
- `ls -l` — long, detailed listing
- `ls -la` — combine them
- `ls -F` — mark folders with `/`, programs with `*`

# Listing things: ls

`ls [options] [path]` — list what is inside a directory.

```
$ ls
Documents  Downloads  code  notes.txt
```

- `ls -a` — also show hidden files (names starting with `.`)
- `ls -l` — long, detailed listing
- `ls -la` — combine them
- `ls -F` — mark folders with `/`, programs with `*`

You can combine options: `ls -la` is the same as `ls -l -a`.

# Reading ls -l

```
-rw-rw-r-- 1 joey joey    21 Jun 19 01:02 notes.txt
drwxrwxr-x 2 joey joey  4096 Jun 19 02:53 code
```

# Reading ls -l

```
-rw-rw-r-- 1 joey joey    21 Jun 19 01:02 notes.txt
drwxrwxr-x 2 joey joey  4096 Jun 19 02:53 code
```

- `-rw-rw-r--` : type + permissions (we'll decode this soon)

# Reading ls -l

```
-rw-rw-r-- 1 joey joey    21 Jun 19 01:02 notes.txt
drwxrwxr-x 2 joey joey  4096 Jun 19 02:53 code
```

- `-rw-rw-r--` : type + permissions (we'll decode this soon)
- `joey joey` : owner and group

# Reading ls -l

```
-rw-rw-r-- 1 joey joey    21 Jun 19 01:02 notes.txt
drwxrwxr-x 2 joey joey  4096 Jun 19 02:53 code
```

- `-rw-rw-r--` : type + permissions (we'll decode this soon)
- `joey joey` : owner and group
- `21 / 4096` : size in bytes

# Reading ls -l

```
-rw-rw-r-- 1 joey joey    21 Jun 19 01:02 notes.txt
drwxrwxr-x 2 joey joey  4096 Jun 19 02:53 code
```

- `-rw-rw-r--` : type + permissions (we'll decode this soon)
- `joey joey` : owner and group
- `21 / 4096` : size in bytes
- the date : last modification time, then the name

# Reading ls -l

```
-rw-rw-r-- 1 joey joey    21 Jun 19 01:02 notes.txt
drwxrwxr-x 2 joey joey  4096 Jun 19 02:53 code
```

- `-rw-rw-r--` : type + permissions (we'll decode this soon)
- `joey joey` : owner and group
- `21 / 4096` : size in bytes
- the date : last modification time, then the name

In Linux, everything is a file — even a directory is a special file.

# Working with Files

# The everyday toolkit

- `mkdir` : make a directory

# The everyday toolkit

- `mkdir` : make a directory
- `touch` : create an empty file

# The everyday toolkit

- `mkdir` : make a directory
- `touch` : create an empty file
- `cp` : copy

# The everyday toolkit

- `mkdir` : make a directory
- `touch` : create an empty file
- `cp` : copy
- `mv` : move *or* rename

# The everyday toolkit

- `mkdir` : make a directory
- `touch` : create an empty file
- `cp` : copy
- `mv` : move *or* rename
- `rm` : remove (careful!)

# The everyday toolkit

- `mkdir` : make a directory
- `touch` : create an empty file
- `cp` : copy
- `mv` : move *or* rename
- `rm` : remove (careful!)

Let's see each one.

# Creating directories and files

```
$ mkdir code
$ mkdir -p web/frontend/styles
$ touch index.html
```

# Creating directories and files

```
$ mkdir code
$ mkdir -p web/frontend/styles
$ touch index.html
```

- `mkdir code` — make one folder

# Creating directories and files

```
$ mkdir code
$ mkdir -p web/frontend/styles
$ touch index.html
```

- `mkdir code` — make one folder
- `mkdir -p web/frontend/styles` — make the whole chain of parents at once

# Creating directories and files

```
$ mkdir code
$ mkdir -p web/frontend/styles
$ touch index.html
```

- `mkdir code` — make one folder
- `mkdir -p web/frontend/styles` — make the whole chain of parents at once
- `touch FILE` — create an empty file

# Copying: cp

```
$ cp notes.txt backup.txt
$ cp -r code code_backup
```

# Copying: cp

```
$ cp notes.txt backup.txt
$ cp -r code code_backup
```

- `cp SRC DEST` — copy a file

# Copying: cp

```
$ cp notes.txt backup.txt
$ cp -r code code_backup
```

- `cp SRC DEST` — copy a file
- `cp -r` — recursive: needed to copy a whole folder

# Copying: cp

```
$ cp notes.txt backup.txt
$ cp -r code code_backup
```

- `cp SRC DEST` — copy a file
- `cp -r` — recursive: needed to copy a whole folder
- `cp -i` — ask before overwriting an existing file

# Moving and renaming: mv

One command does both — it depends on the destination.

# Moving and renaming: mv

One command does both — it depends on the destination.

```
$ mv old.txt new.txt        # rename
$ mv report.txt Documents/  # move into a folder
```

# Moving and renaming: mv

One command does both — it depends on the destination.

```
$ mv old.txt new.txt          # rename
$ mv report.txt Documents/    # move into a folder
```

- If the destination is a directory ⇒ the file is moved inside it.

# Moving and renaming: mv

One command does both — it depends on the destination.

```
$ mv old.txt new.txt          # rename
$ mv report.txt Documents/    # move into a folder
```

- If the destination is a directory ⇒ the file is moved inside it.
- If the destination doesn't exist ⇒ it's a rename.

# Moving and renaming: mv

One command does both — it depends on the destination.

```
$ mv old.txt new.txt          # rename
$ mv report.txt Documents/    # move into a folder
```

- If the destination is a directory ⇒ the file is moved inside it.
- If the destination doesn't exist ⇒ it's a rename.
- Careful: if the destination is an existing file, it gets overwritten.

# Moving and renaming: mv

One command does both — it depends on the destination.

```
$ mv old.txt new.txt          # rename
$ mv report.txt Documents/    # move into a folder
```

- If the destination is a directory ⇒ the file is moved inside it.
- If the destination doesn't exist ⇒ it's a rename.
- Careful: if the destination is an existing file, it gets overwritten.
- `mv -i` prompts before overwriting.

# Removing: rm (handle with care)

```
$ rm notes.txt
$ rm -r old_folder
```

# Removing: rm (handle with care)

```
$ rm notes.txt
$ rm -r old_folder
```

- `rm FILE` — delete a file. There is no Recycle Bin. No undo.

# Removing: rm (handle with care)

```
$ rm notes.txt
$ rm -r old_folder
```

- `rm FILE` — delete a file. There is no Recycle Bin. No undo.
- `rm -r` — recursive: needed to delete a folder

# Removing: rm (handle with care)

```
$ rm notes.txt
$ rm -r old_folder
```

- `rm FILE` — delete a file. There is no Recycle Bin. No undo.
- `rm -r` — recursive: needed to delete a folder
- `rm -i` — ask before each deletion (a good safety net)

# Removing: rm (handle with care)

```
$ rm notes.txt
$ rm -r old_folder
```

- `rm FILE` — delete a file. There is no Recycle Bin. No undo.
- `rm -r` — recursive: needed to delete a folder
- `rm -i` — ask before each deletion (a good safety net)
- `rm -f` — force, never ask

# Removing: rm (handle with care)

```
$ rm notes.txt
$ rm -r old_folder
```

- `rm FILE` — delete a file. There is no Recycle Bin. No undo.
- `rm -r` — recursive: needed to delete a folder
- `rm -i` — ask before each deletion (a good safety net)
- `rm -f` — force, never ask

Never run `sudo rm -rf /`. It tries to erase the entire system.

# Exercise: try it yourself

1. Create four files: `a.java`, `a.js`, `a.html`, `a.css`.

# Exercise: try it yourself

1. Create four files: `a.java`, `a.js`, `a.html`, `a.css`.
2. Make a folder `web_project` with subfolders `backend`, `frontend`, `styles`.

# Exercise: try it yourself

1. Create four files: `a.java`, `a.js`, `a.html`, `a.css`.
2. Make a folder `web_project` with subfolders `backend`, `frontend`, `styles`.
3. Move the `.java` file to `backend`, the `.js` and `.html` files to `frontend`, and the `.css` file to `styles`.

# Exercise: try it yourself

1. Create four files: `a.java`, `a.js`, `a.html`, `a.css`.
2. Make a folder `web_project` with subfolders `backend`, `frontend`, `styles`.
3. Move the `.java` file to `backend`, the `.js` and `.html` files to `frontend`, and the `.css` file to `styles`.
4. Use `ls -R web_project` to check your work.

# Exercise: try it yourself

1. Create four files: `a.java`, `a.js`, `a.html`, `a.css`.
2. Make a folder `web_project` with subfolders `backend`, `frontend`, `styles`.
3. Move the `.java` file to `backend`, the `.js` and `.html` files to `frontend`, and the `.css` file to `styles`.
4. Use `ls -R web_project` to check your work.

Hint: `mv a.js a.html frontend/` moves both at once.

# Looking Inside Files

# Viewing file contents

- `cat FILE` — dump the whole file to the screen

# Viewing file contents

- `cat FILE` — dump the whole file to the screen
- `less FILE` — view a long file one screen at a time

# Viewing file contents

- `cat FILE` — dump the whole file to the screen
- `less FILE` — view a long file one screen at a time
- `head FILE` / `tail FILE` — first / last 10 lines

# Viewing file contents

- `cat FILE` — dump the whole file to the screen
- `less FILE` — view a long file one screen at a time
- `head FILE` / `tail FILE` — first / last 10 lines
- `wc FILE` — count lines, words, characters

# Viewing file contents

- `cat FILE` — dump the whole file to the screen
- `less FILE` — view a long file one screen at a time
- `head FILE` / `tail FILE` — first / last 10 lines
- `wc FILE` — count lines, words, characters

```
$ wc notes.txt
  3  12  74 notes.txt
```

3 lines, 12 words, 74 characters in `notes.txt`.

# Viewing file contents

- `cat FILE` — dump the whole file to the screen
- `less FILE` — view a long file one screen at a time
- `head FILE` / `tail FILE` — first / last 10 lines
- `wc FILE` — count lines, words, characters

```
$ wc notes.txt
  3  12  74 notes.txt
```

3 lines, 12 words, 74 characters in `notes.txt`.
Inside `less`: arrows to scroll, `q` to quit.

# Searching inside files: grep

`grep PATTERN FILE` — print lines that contain the pattern.

# Searching inside files: grep

`grep PATTERN FILE` — print lines that contain the pattern.

```
$ grep "error" logfile.txt
$ grep -i "hello" notes.txt   # ignore upper/lower case
$ grep -n "TODO" main.c       # show line numbers
```

# Searching inside files: grep

`grep PATTERN FILE` — print lines that contain the pattern.

```
$ grep "error" logfile.txt
$ grep -i "hello" notes.txt    # ignore upper/lower case
$ grep -n "TODO" main.c        # show line numbers
```

- `-i` : case-insensitive

# Searching inside files: grep

`grep PATTERN FILE` — print lines that contain the pattern.

```
$ grep "error" logfile.txt
$ grep -i "hello" notes.txt    # ignore upper/lower case
$ grep -n "TODO" main.c        # show line numbers
```

- `-i` : case-insensitive
- `-n` : show line numbers

# Searching inside files: grep

`grep PATTERN FILE` — print lines that contain the pattern.

```
$ grep "error" logfile.txt
$ grep -i "hello" notes.txt    # ignore upper/lower case
$ grep -n "TODO" main.c        # show line numbers
```

- `-i` : case-insensitive
- `-n` : show line numbers
- `-r` : search recursively through a whole folder

# Searching inside files: grep

`grep PATTERN FILE` — print lines that contain the pattern.

```
$ grep "error" logfile.txt
$ grep -i "hello" notes.txt    # ignore upper/lower case
$ grep -n "TODO" main.c        # show line numbers
```

- `-i` : case-insensitive
- `-n` : show line numbers
- `-r` : search recursively through a whole folder
- `-v` : invert — show lines that do not match

# Finding files: find

`grep` looks *inside* files. `find` looks *for* files.

# Finding files: find

`grep` looks *inside* files. `find` looks *for* files.

```
$ find . -name "notes.txt"
$ find . -name "*.txt"
$ find /home -type d -name "code"
```

# Finding files: find

`grep` looks *inside* files. `find` looks *for* files.

```
$ find . -name "notes.txt"
$ find . -name "*.txt"
$ find /home -type d -name "code"
```

- first argument is where to start (`.` = here); it searches recursively

# Finding files: find

`grep` looks *inside* files. `find` looks *for* files.

```
$ find . -name "notes.txt"
$ find . -name "*.txt"
$ find /home -type d -name "code"
```

- first argument is where to start (`.` = here); it searches recursively
- `-name` : match by name (`*` is a wildcard)

# Finding files: find

`grep` looks *inside* files. `find` looks *for* files.

```
$ find . -name "notes.txt"
$ find . -name "*.txt"
$ find /home -type d -name "code"
```

- first argument is where to start (`.` = here); it searches recursively
- `-name` : match by name (`*` is a wildcard)
- `-type f` : only files  `-type d` : only directories

# Permissions and sudo

# Why permissions exist

- Many people (and programs) can share one Linux machine.

# Why permissions exist

- Many people (and programs) can share one Linux machine.
- Each file controls who can read, write, and execute it.

# Why permissions exist

- Many people (and programs) can share one Linux machine.
- Each file controls who can read, write, and execute it.
- This keeps your files private and the system safe.

# Why permissions exist

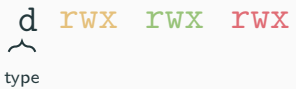
- Many people (and programs) can share one Linux machine.
- Each file controls who can read, write, and execute it.
- This keeps your files private and the system safe.

Three actions: r read, w write, x execute.

# Decoding the permission string

```
d rwx rwx rwx
```

# Decoding the permission string



# Decoding the permission string



# Decoding the permission string



# Decoding the permission string



# Decoding the permission string




- First letter: `d` = directory, `-` = regular file, `l` = link.

# Decoding the permission string




- First letter: `d` = directory, `-` = regular file, `l` = link.
- Then three groups of `rwx` for owner, group, and everyone else.

# Permissions on directories mean something different

- r on a directory: you can `ls` (list) its contents.

# Permissions on directories mean something different

- r on a directory: you can `ls` (list) its contents.
- w on a directory: you can create / delete entries in it.

# Permissions on directories mean something different

- r on a directory: you can `ls` (list) its contents.
- w on a directory: you can create / delete entries in it.
- x on a directory: you can `cd` into it.

# Permissions on directories mean something different

- r on a directory: you can `ls` (list) its contents.
- w on a directory: you can create / delete entries in it.
- x on a directory: you can `cd` into it.

So a folder usually needs `r-x` just to be usable.

# Changing permissions: chmod

Two ways to write the same thing.

# Changing permissions: chmod

Two ways to write the same thing.

- **Symbolic:** who ( `u` / `g` / `o` / `a` ) + or − + `rwx`

# Changing permissions: chmod

Two ways to write the same thing.

- **Symbolic:** who ( `u` / `g` / `o` / `a` ) + or − + `rwx`

```
$ chmod u+x script.sh      # let the owner execute it
$ chmod go-w notes.txt     # remove write for group & others
```

# Changing permissions: chmod

Two ways to write the same thing.

- **Symbolic:** who ( `u` / `g` / `o` / `a` ) + or − + `rwx`

```
$ chmod u+x script.sh      # let the owner execute it
$ chmod go-w notes.txt     # remove write for group & others
```

- **Octal:** each `rwx` group becomes a number ( `r=4, w=2, x=1` )

# Changing permissions: chmod

Two ways to write the same thing.

- **Symbolic:** who ( `u` / `g` / `o` / `a` ) + or − + `rwx`

```
$ chmod u+x script.sh      # let the owner execute it
$ chmod go-w notes.txt     # remove write for group & others
```

- **Octal:** each `rwx` group becomes a number ( `r=4, w=2, x=1` )

```
$ chmod 755 script.sh
# 7=rwx (owner)  5=r-x (group)  5=r-x (others)
```

# Users, root, and sudo

- root is the all-powerful superuser (can do anything).

# Users, root, and sudo

- root is the all-powerful superuser (can do anything).
- Your normal user is deliberately limited — for safety.

# Users, root, and sudo

- root is the all-powerful superuser (can do anything).
- Your normal user is deliberately limited — for safety.
- `sudo CMD` runs one command as root, just for that command.

# Users, root, and sudo

- root is the all-powerful superuser (can do anything).
- Your normal user is deliberately limited — for safety.
- `sudo CMD` runs one command as root, just for that command.
- You'll be asked for your password the first time.

# Users, root, and sudo

- root is the all-powerful superuser (can do anything).
- Your normal user is deliberately limited — for safety.
- `sudo CMD` runs one command as root, just for that command.
- You'll be asked for your password the first time.



# Users, root, and sudo

- root is the all-powerful superuser (can do anything).
- Your normal user is deliberately limited — for safety.
- `sudo CMD` runs one command as root, just for that command.
- You'll be asked for your password the first time.

With great power comes great responsibility — think before you `sudo`.



# The power (and danger) of sudo

```
$ ifconfig -a          # bash: ifconfig: command not found
$ sudo ifconfig -a     # works: runs as root
```

# The power (and danger) of sudo

```
$ ifconfig -a          # bash: ifconfig: command not found
$ sudo ifconfig -a     # works: runs as root
```

- Installing software, editing system files, etc. all need `sudo`.

# The power (and danger) of sudo

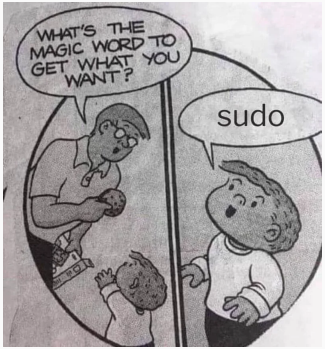
```
$ ifconfig -a          # bash: ifconfig: command not found
$ sudo ifconfig -a     # works: runs as root
```

- Installing software, editing system files, etc. all need `sudo`.
- `chown user FILE` / `chgrp group FILE` change owner / group.

# The power (and danger) of sudo

```
$ ifconfig -a          # bash: ifconfig: command not found
$ sudo ifconfig -a     # works: runs as root
```

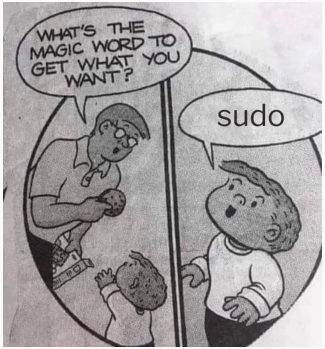
- Installing software, editing system files, etc. all need `sudo`.
- `chown user FILE` / `chgrp group FILE` change owner / group.



# The power (and danger) of sudo

```
$ ifconfig -a          # bash: ifconfig: command not found
$ sudo ifconfig -a     # works: runs as root
```

- Installing software, editing system files, etc. all need `sudo`.
- `chown user FILE` / `chgrp group FILE` change owner / group.
- Only use `sudo` when you actually understand the command.



# Redirection & Pipes

# Redirecting output

By default, output goes to the screen. You can send it to a file instead.

# Redirecting output

By default, output goes to the screen. You can send it to a file instead.

```
$ echo "hello" > greeting.txt     # write (overwrites!)
$ echo "world" >> greeting.txt    # append (adds a line)
$ sort < names.txt                # read input from a file
```

# Redirecting output

By default, output goes to the screen. You can send it to a file instead.

```
$ echo "hello" > greeting.txt     # write (overwrites!)
$ echo "world" >> greeting.txt    # append (adds a line)
$ sort < names.txt                # read input from a file
```

- `>` : send output to a file (replaces existing content)

# Redirecting output

By default, output goes to the screen. You can send it to a file instead.

```
$ echo "hello" > greeting.txt     # write (overwrites!)
$ echo "world" >> greeting.txt    # append (adds a line)
$ sort < names.txt                # read input from a file
```

- `>` : send output to a file (replaces existing content)
- `>>` : append to the end of a file

# Redirecting output

By default, output goes to the screen. You can send it to a file instead.

```
$ echo "hello" > greeting.txt     # write (overwrites!)
$ echo "world" >> greeting.txt    # append (adds a line)
$ sort < names.txt                # read input from a file
```

- `>` : send output to a file (replaces existing content)
- `>>` : append to the end of a file
- `<` : take input from a file

# Pipes: connecting commands

A pipe `|` feeds the output of one command into the next.



# Pipes: connecting commands

A pipe `|` feeds the output of one command into the next.



# Pipes: connecting commands

A pipe `|` feeds the output of one command into the next.



# Pipes: connecting commands

A pipe `|` feeds the output of one command into the next.




- `|` chains commands; `>>` writes to a *file*. Different jobs!

# Pipes in action

```
$ history | grep "cd"
$ ls /bin /usr/bin | sort | uniq | less
```

# Pipes in action

```
$ history | grep "cd"
$ ls /bin /usr/bin | sort | uniq | less
```

- First: search your command history for every time you used `cd`.

# Pipes in action

```
$ history | grep "cd"
$ ls /bin /usr/bin | sort | uniq | less
```

- First: search your command history for every time you used `cd`.
- Second: list two folders, sort them, drop duplicates, and view one screen at a time.

# Pipes in action

```
$ history | grep "cd"
$ ls /bin /usr/bin | sort | uniq | less
```

- First: search your command history for every time you used `cd`.
- Second: list two folders, sort them, drop duplicates, and view one screen at a time.



# Pipes in action

```
$ history | grep "cd"
$ ls /bin /usr/bin | sort | uniq | less
```

- First: search your command history for every time you used `cd`.
- Second: list two folders, sort them, drop duplicates, and view one screen at a time.

Small commands + pipes = powerful combinations.



# Installation & Help

# Installing software with apt

Some Ubuntu programs can be installed through `apt` (its package manager).

# Installing software with apt

Some Ubuntu programs can be installed through `apt` (its package manager).

```
$ sudo apt update          # refresh the package list
$ sudo apt install htop    # install a program
$ sudo apt remove htop     # remove it
```

# Installing software with apt

Some Ubuntu programs can be installed through `apt` (its package manager).

```
$ sudo apt update          # refresh the package list
$ sudo apt install htop    # install a program
$ sudo apt remove htop     # remove it
```

- Always `update` first so you get current versions.

# Installing software with apt

Some Ubuntu programs can be installed through `apt` (its package manager).

```
$ sudo apt update          # refresh the package list
$ sudo apt install htop    # install a program
$ sudo apt remove htop     # remove it
```

- Always `update` first so you get current versions.
- No hunting for installers online — it's all one command.

# Aliases: shortcuts you invent

An alias is your own short name for a longer command.

# Aliases: shortcuts you invent

An alias is your own short name for a longer command.

```
$ alias ll='ls -alF'
$ alias rm='rm -i'      # always ask before deleting
```

# Aliases: shortcuts you invent

An alias is your own short name for a longer command.

```
$ alias ll='ls -alF'
$ alias rm='rm -i'      # always ask before deleting
```

- Aliases set in the terminal vanish when you close it.

# Aliases: shortcuts you invent

An alias is your own short name for a longer command.

```
$ alias ll='ls -alF'
$ alias rm='rm -i'      # always ask before deleting
```

- Aliases set in the terminal vanish when you close it.
- Put them in `~/.bash_aliases` to make them permanent.

# Aliases: shortcuts you invent

An alias is your own short name for a longer command.

```
$ alias ll='ls -alF'
$ alias rm='rm -i'      # always ask before deleting
```

- Aliases set in the terminal vanish when you close it.
- Put them in `~/.bash_aliases` to make them permanent.

`alias rm='rm -i'` is a classic safety habit.


$ alias cd='rm -rf'
38

# When you forget a command

- `CMD --help` — quick summary of options

# When you forget a command

- `CMD --help` — quick summary of options
- `man CMD` — the full manual (press `q` to quit)

# When you forget a command

- `CMD --help` — quick summary of options
- `man CMD` — the full manual (press `q` to quit)
- `tldr CMD` — short, example-driven help (the friendly one)

# When you forget a command

- `CMD --help` — quick summary of options
- `man CMD` — the full manual (press `q` to quit)
- `tldr CMD` — short, example-driven help (the friendly one)
- `which CMD` — where is this command located?

# When you forget a command

- `CMD --help` — quick summary of options
- `man CMD` — the full manual (press `q` to quit)
- `tldr CMD` — short, example-driven help (the friendly one)
- `which CMD` — where is this command located?

You don't need to memorise everything — you need to know how to look it up.

# Hard Reality

